

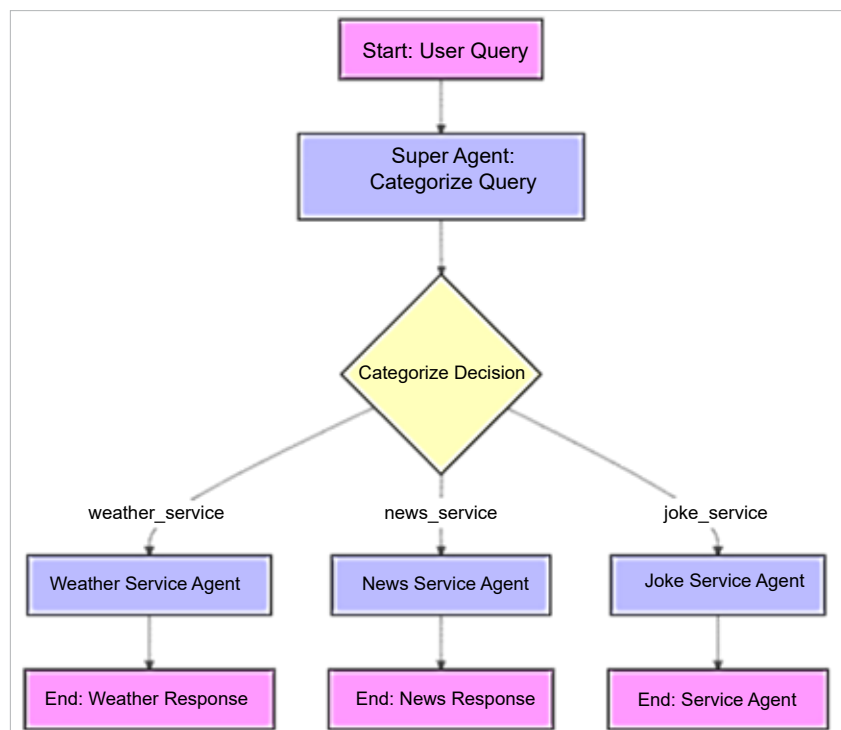


Figure 1: The flow diagram

```

from langchain.schema import SystemMessage
from langgraph.graph import StateGraph
from pydantic import BaseModel
import os

```

Now, we set up the Hugging Face API key. This sets the environment variable for your Hugging Face API token so that you can interact with the model. Replace *YOUR KEY HERE* with actual *API KEY*.

```
os.environ["HUGGINGFACEHUB_API_TOKEN"] = "YOUR KEY HERE"
```

Next, a specific Hugging Face model (Mistral-7B-Instruct-v0.1) is loaded. *temperature* controls randomness, and *max_length* controls the length of the generated responses.

```

llm = HuggingFaceHub(
    repo_id="mistralai/Mistral-7B-Instruct-v0.1", # Public
    model
    model_kwargs={"temperature": 0.7, "max_length": 256}
)

```

The state schema defines a structured model for the state of the system, which includes:

```
input_data: The user query.
```

```

category: The category of the query
(e.g., weather, news, joke).
response: The response generated by the
agent (e.g., weather forecast, news,
joke).

```

```

class ExpertSystemState(BaseModel):
    input_data: str # User query
    category: str = None # Categorized
    question type
    response: str = None # AI-generated
    response

```

The 'helper' function logs events during the workflow (like analysing a query, classifying a query, etc) for debugging or tracking purposes.

```

def log_event(step: str, message: str):
    print(f"[{step}] {message}")

```

Defining the service agents

Each agent corresponds to a different category of query. Here's how each agent works.

- **Super Agent** (*super_agent* function):
 - Analyses the user's query and categorises it (weather, news, or joke).
 - Uses the Hugging Face model to classify the query based on the prompt.
- **Weather Service** (*weather_service* function):
 - Provides weather-related information based on the query.
- **News Service** (*news_service* function):
 - Provides the latest news based on the query.
- **Joke Service** (*joke_service* function):
 - Provides a joke based on the query.

Each of these functions generates a response using the Hugging Face model, logs it, and returns the *ExpertSystemState* with the appropriate response.

```

def super_agent(state: ExpertSystemState) ->
ExpertSystemState:
    log_event("Super Agent", f"Analyzing query: {state.
input_data}")

    prompt = f"""

```

You are a Super Agent tasked with categorizing the query into one of the following categories:

```
- weather_service
```